DIP - Ultra-Short Python Lab Programs

Exam-friendly versions for quick memorization and demonstration

5) RGB, CMY and HSI Color Components

Python 3 + OpenCV + NumPy

```
import cv2,numpy as np
im=cv2.imread("image.jpg"); b,g,r=cv2.split(im); z=np.zeros_like(b)
cv2.imshow("Red",cv2.merge([z,z,r])); cv2.imshow("Green",cv2.merge([z,g,z]))
cv2.imshow("Blue",cv2.merge([b,z,z])); cv2.imshow("Cyan",cv2.merge([255-r,255-r,z]))
cv2.imshow("Magenta",cv2.merge([255-g,z,255-g])); cv2.imshow("Yellow",cv2.merge([z,255-b,255-b]))

R,G,B=r/255,g/255,b/255; I=(R+G+B)/3
S=1-3*np.minimum(R,np.minimum(G,B))/(R+G+B+1e-6)
x=np.sqrt((R-G)**2+(R-B)*(G-B))+1e-6
H=np.arccos(np.clip((2*R-G-B)/(2*x),-1,1)); H=np.where(B>G,2*np.pi-H,H)

h=(H*28.5).astype(np.uint8); s=(S*255).astype(np.uint8); i=(I*255).astype(np.uint8)
cv2.imshow("Hue",cv2.cvtColor(cv2.merge([h,np.full_like(h,255),np.full_like(h,255)]),cv2.COLOR_HSV2BGR))
cv2.imshow("Saturation",cv2.cvtColor(cv2.merge([h,s,np.full_like(h,255)]),cv2.COLOR_HSV2BGR))
cv2.imshow("Intensity",cv2.applyColorMap(i,cv2.COLORMAP_JET))
cv2.waitKey(0)
```

Key memory: BGR -> split -> RGB/CMY -> HSI (H, S, I).

11) Contrast, Threshold, Correlation and Convolution

Python 3 + OpenCV + NumPy

```
import cv2
import numpy as np
im=cv2.imread("image.jpg",0)
contrast=cv2.normalize(im,None,0,255,cv2.NORM_MINMAX)
_,threshold=cv2.threshold(im,127,255,cv2.THRESH_BINARY)
k=np.ones((3,3),np.float32)/9
cor=cv2.filter2D(im,-1,k)
conv=cv2.filter2D(im,-1,k[::-1,::-1])
cv2.imshow("Contrast",contrast)
cv2.imshow("Threshold",threshold)
cv2.imshow("Correlation",cor)
cv2.imshow("Convolution",conv)
cv2.waitKey(0)
```

Key memory: normalize -> threshold -> kernel -> filter2D.

15) Spatial Correlation and Convolution using Custom Kernel

Python 3 + OpenCV + NumPy

```
import cv2
import numpy as np
im=cv2.imread("image.jpg",0)
k=np.array([[1,2,0],
            [0,1,0],
            [0,0,0]],np.float32)/4
cor=cv2.filter2D(im,-1,k)
conv=cv2.filter2D(im,-1,k[::-1,::-1])
cv2.imshow("Correlation",cor)
cv2.imshow("Convolution",conv)
cv2.waitKey(0)
```

Key memory: correlation = k; convolution = flipped k.

23) Morphological Operations

Python 3 + OpenCV + NumPy

```
import cv2
import numpy as np
im=cv2.imread("image.jpg",0)
_,im=cv2.threshold(im,127,255,cv2.THRESH_BINARY)
k=np.ones((3,3),np.uint8)
d=cv2.dilate(im,k)
e=cv2.erode(im,k)
o=cv2.morphologyEx(im,cv2.MORPH_OPEN,k)
c=cv2.morphologyEx(im,cv2.MORPH_CLOSE,k)
cv2.imshow("Original",im)
cv2.imshow("Dilation",d)
cv2.imshow("Erosion",e)
cv2.imshow("Opening",o)
cv2.imshow("Closing",c)
cv2.waitKey(0)
```

Key memory: binary -> structuring element -> D, E, Opening, Closing.

24) Hit-or-Miss Transformation

Python 3 + OpenCV + NumPy

```
import cv2
import numpy as np
im=cv2.imread("image.jpg",0)
_,im=cv2.threshold(im,127,255,cv2.THRESH_BINARY)
k=np.array([[1,1,0],
            [1,-1,0],
            [0,0,0]],np.int8)
out=cv2.morphologyEx(im,cv2.MORPH_HITMISS,k)
cv2.imshow("Original",im)
cv2.imshow("Hit-or-Miss",out)
cv2.waitKey(0)
```

Key memory: binary -> kernel -> MORPH_HITMISS.

Install Libraries

Run this command in the terminal:

```
pip install opencv-python numpy
```

Required libraries: OpenCV (opencv-python) and NumPy. No other pip packages are required for these programs.

Note: Program 5 uses true HSI formulas for Hue, Saturation and Intensity. Programs 11 and 15 use spatial filtering with custom kernels. Program 23 uses a 3x3 structuring element. Program 24 uses a binary image for hit-or-miss detection.